组培苗生产管理信息系统

软件设计文档

Plant Tissue Culture Management Information System

Software Design Document

文档编号：PTCMIS-SDD-001

版本：V1.0

编制：陈慧然

日期：2026 年 10 月 30 日

目　　录

1 引言 1
1.1 编写目的 1
1.2 读者对象 1
1.3 术语与缩略 1
1.4 参考文档 2
1.5 文档结构 2
2 设计概述 3
2.1 设计目标 3
2.2 总体架构 3
2.3 技术栈与版本 4
2.4 分层结构与包职责 5
2.5 关键设计决策 5
3 后端设计 6
3.1 请求处理链路 6
3.2 接口层设计 7
3.3 业务层设计 7
3.4 持久层设计 8
3.5 缓存与序列 9
3.6 统一异常与响应 9
3.7 事务与并发 9
3.8 状态机实现 10
4 数据设计 10
4.1 数据模型总览 10
4.2 数据表清单 11
4.3 核心表结构 12
4.4 编码与枚举 14
4.5 键、约束与索引 14
4.6 冗余与水化 15
4.7 容量与大表治理 15
5 接口设计 16
5.1 协议与地址规范 16
5.2 请求与响应模型 16
5.3 鉴权与权限 17
5.4 错误码与提示 17
5.5 主要接口 17
5.6 打印通道 18
5.7 扫码、附件与导出 18
6 前端设计 19
6.1 技术形态 19
6.2 目录结构 19
6.3 数据层设计 20
6.4 界面清单 20
6.5 交互与校验约定 20
6.6 打印集成 21
6.7 构建与性能 21
7 打印子系统设计 21
7.1 进程与通信模型 21
7.2 报文约定 22

7.3 指令装配 22
7.4 平台适配 22
7.5 错误处理 23
7.6 构建与分发 23
8 安全设计 23
8.1 认证与会话 23
8.2 授权模型 24
8.3 口令与令牌的存储 24
8.4 输入校验与防护 24
8.5 审计与日志 24
9 部署与运维 25
9.1 部署拓扑 25
9.2 配置项 25
9.3 构建与发布 26
9.4 备份与恢复 26
9.5 监控与排障 26
10 设计约束 26
10.1 设计约束 26
附录 A 数据表清单 27
附录 B 接口清单 28

1 引言

1.1 编写目的

本文档依据 PTCMIS 现有源码，说明组培苗生产管理信息系统的软件结构、模块划分、数据组织、接口约定与关键实现策略，作为开发维护、测试设计与上线运维的共同依据。文档面向已阅读《需求规格说明书》的读者，不重复业务背景，只回答“如何实现”。

文中所有结构与规则均以源码为准：类名、表名、字段名、枚举取值、接口路径按实际代码书写，便于按图索骥。

1.2 读者对象

后端开发人员依据第 3、4、5 章理解分层职责、数据模型与接口约定；前端开发人员依据第 6 章理解目录分层、数据服务与界面组织；负责标签打印的工程人员依据第 7 章理解进程协议与平台适配；运维与安全人员依据第 8、9 章理解鉴权模型、部署拓扑与运维动作。

评审人员可将第 2 章的设计决策表与《需求规格说明书》的条款对照，核对实现与需求的对应关系。

1.3 术语与缩略

本文档沿用《需求规格说明书》表 1 的业务术语，另补充下表的技术术语。

表 1 技术术语与缩略

术语	说明
Action	后端控制器类，标注 @RestController 与 @RequestMapping，负责接收请求、参数绑定与返回结果，不承载业务判断
Manager	后端业务层类，承载校验、状态流转、事务边界与跨表编排
Repository / DAO	持久层接口与实现，基于 JPA 与 EntityManager 拼装 CriteriaQuery
PO	持久化对象，与数据库表一一对应，位于 stubs/entities 包，字段注释中标注列名与 DDL
Model	接口出入参模型，位于 stubs/models 包，与 PO 分离，避免直接暴露表结构
Token	登录令牌，由服务端生成并保存在进程内存，经请求头 x-ptcmis-token 传递
闭包表	以祖先—后代—距离三元组记录层级关系的数据结构，此处用于培养瓶代际追溯
LODOP	浏览器端打印控件，通过 ActiveX 或插件驱动本地打印机
TSPL	TSC 系列标签打印机指令集，标签内容以文本指令下发
Zoneless	Angular 无 zone.js 的变更检测模式，依赖信号量显式触发刷新

1.4 参考文档

表 2 参考文档

编号	名称	用途
D1	组培苗生产管理信息系统 需求规格说明书（PTCMIS-SRS-001）	业务规则、功能条目与非功能指标的来源
D2	ptcmis-server 源码（Spring Boot 工程）	分层结构、业务校验、接口与数据访问的实现依据
D3	ptcmis-web 源码（Angular 工程）	界面组织、数据服务与打印集成的实现依据
D4	ptcmis-printer 源码（C++17 工程）	本地打印进程的协议、指令与平台适配依据
D5	baseline.sql 与各 PO 注释	表结构、字段类型与默认值的核对依据

1.5 文档结构

第 2 章给出总体架构与技术选型，并列出影响全局的设计决策；第 3 章说明后端分层、异常、事务与状态机实现；第 4 章说明数据模型、键与索引、冗余设计；第 5 章说明接口协议与端点分布；第 6 章说明前端分层与界面组织；第 7 章说明本地打印子系统的进程模型与协议；第 8 章说明认证、授权与安全防护的实现现状；第 9 章说明部署与运维；第 10 章列出设计约束；附录 A、B 分别给出数据表与接口的完整清单。

2 设计概述

2.1 设计目标

系统围绕“一瓶一码、一码到底”组织：每个培养瓶在诞生时取得全局唯一瓶码，此后每一次接种、出库、入库、质检、调差都挂在瓶码上，形成可回溯的瓶级台账。设计上服务于四个目标。

- 口径唯一：瓶数与株数分离记账，出库记负、入库记正，任何库存数字都能由流水重算，报表不另立口径。
- 操作可逆：任务与出入库均设有撤销与驳回路径，撤销前先校验下游是否已产生业务，避免出现半成品数据。
- 现场友好：关键路径以扫码为主，一次扫码即可定位瓶与任务，界面不要求人工查找。
- 部署轻量：服务端单实例、单库部署，依赖少、启动快，可在厂区内独立运行。

2.2 总体架构

系统由三个可独立构建的工程组成：后端服务 ptcmis-server 提供全部业务接口；前端应用 ptcmis-web 以静态资源形式由 Nginx 托管，通过 HTTP 调用后端；本地打印进程 ptcmis-printer 常驻客户端，由浏览器经本地通道下发打印指令。三者之间只有两处强耦合：请求头中的令牌，以及打印指令的 JSON 行协议。

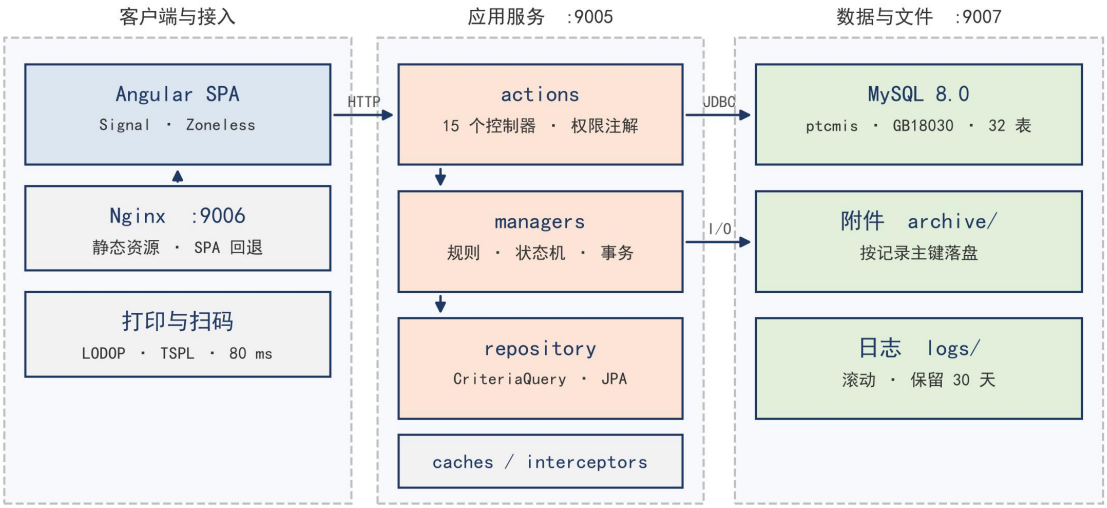


图 1 系统总体架构

架构上遵守两条边界：后端不直接驱动打印机，打印能力全部落在客户端，使服务端可在无外设的服务器上运行；前端不直接访问数据库，所有读写经接口完成，界面只持有展示态与表单态。

2.3 技术栈与版本

表 3 技术栈与版本

层次	技术选型	版本	说明
后端框架	Spring Boot	4.0.7	Web、依赖注入、JPA、事务管理由框架提供
后端语言	Java	21	编译目标为 21，可使用记录类与模式匹配
持久层	JPA + EntityManager	随框架	以 CriteriaQuery 拼装查询，避免大量拼接 SQL
数据库	MySQL	8.0	库名 ptcmis，字符集 GB18030
前端框架	Angular	22.1.2	独立组件、路由、表单与 HTTP 客户端
前端 UI	Angular Material	22.1.2	表格、对话框、表单控件与主题
图表	ECharts	6.1.0	统计报表的柱线图与占比图
前端工具库	lodash-es、rxjs、uuid	4.18 / 7.8 / 14	集合处理、流式请求与本地唯一标识
Web 服务	Nginx	—	静态资源托管、SPA 回退、gzip 与缓存
打印进程	C++ / CMake	C++17 / 3.18+	WinSpool 与 CUPS 双平台，输出可独立分发可执行文件
打印指令	TSPL / TSS24.BF2	—	标签尺寸、间隙、方向、二维码与文本指令

2.4 分层结构与包职责

后端按“接口层—业务层—持久层”三层组织，包名与职责一一对应。分层的目的不是形式，而是把可变的业务规则集中到业务层，使接口层只做协议转换、持久层只做数据存取。

表 4 后端包结构与职责

包	主要类	职责	约束
actions	TissueTaskAction、StorageInOutAction、BottleAction 等 15 个控制器	路由映射、参数绑定、权限注解、结果封装	不写业务判断，不直接操作 EntityManager
managers	TissueTaskManager、StorageInOutManager、BottleManager、BottleCodeSequenceManager、PassportManager 等	业务校验、状态流转、跨表编排、事务边界	所有写操作在此开启事务，异常以中文消息上抛
repository	各 PO 对应的 DAO 与查询实现	CriteriaQuery 组装、分页、批量写入、统计聚合	只返回实体或投影，不做业务加工
caches	字典与令牌相关缓存	把低频变更的字典项与在线令牌放入内存	进程内缓存，重启即失效，不做多实例同步
interceptors	AuthInterceptor、ExceptionsHandler	统一鉴权与统一异常映射	鉴权只校验令牌有效性，角色判断交给注解
stubs/entities	32 个 PO	表结构映射，字段注释含列名与 DDL	不得在 PO 上写业务方法
stubs/models	请求与响应模型、QueryCondition、QueryResultPaged	接口契约	与 PO 分离，避免表结构外泄
utils	批次校验、编号生成、导出等工具	无状态函数	不依赖容器上下文

2.5 关键设计决策

下列决策决定了系统的主要形态，改动其中任何一条都会引起跨层调整。

表 5 关键设计决策

编号	决策	理由	影响
K1	单库单实例，不引入分库分表与分布式事务	厂区规模有限，瓶级数据三年内可控；单库便于备份与迁移	横向扩展受限，大表依赖索引与归档
K2	令牌保存在服务进程内存	实现简单，无需额外会话存储	服务重启后全员重新登录，无法多实例部署
K3	业务数量一律以整数存储，出库记负、入库记正	库存可由流水求和得到，避免方向字段与正负号双重表达	汇总时必须区分业务类型，误写符号会污染台账
K4	瓶码前缀由服务端统一发号，序列加悲观锁	前缀必须全局唯一且不可复用，锁的开销远小于冲突代价	发号成为写入热点，需控制单次申请数量
K5	代际关系用闭包表存储	支持任意层级上下追溯，查询为一次索引扫描	随瓶数增长行数放大，需限制追溯深度
K6	接口统一 POST，出入参统一 JSON	现场网络环境简单，避免 GET 参数编码差异；便于统一拦截与审计	接口非 REST 风格，缓存与幂等由业务层约定
K7	打印能力放在客户端，服务端只记录打印结果	服务端无需接触打印机驱动，部署位置自由	客户端需安装打印控件或本地进程，版本需与前端一致
K8	批量写入按 500 条一批 flush 与 clear	控制一级缓存规模，避免大批量导入时内存膨胀	批次边界即提交边界，失败后需整批重试

3 后端设计

3.1 请求处理链路

一次业务请求依次经过：Nginx 转发 → AuthInterceptor 校验令牌 → Action 绑定参数并校验权限注解 → Manager 执行业务校验与状态流转 → Repository 读写数据库 → Action 封装响应。异常在任意环节抛出后由 ExceptionsHandler 统一转换为 HTTP 状态码与中文消息，前端据此提示。

链路中设置两条纪律：其一，鉴权只判断“令牌是否有效、账号是否启用”，具体角色由 Action 方法上的权限注解声明，避免在拦截器里堆积业务分支；其二，业务校验一律前置于写操作，先校验后落库，使失败请求不留下半成品数据。

3.2 接口层设计

15 个控制器按业务域划分，端点总数 116 个，全部以 POST 暴露，路径统一以 /api 开头。控制器方法只做三件事：取当前用户、组装查询条件、调用业务层并返回。

表 6 控制器与端点分布

控制器	端点数	职责	典型端点
StorageInOutAction	26	出入库单据的申请、备出、接收、入库、驳回、作废与查询	/api/storageInOut/outProcessForTask
BottleAction	18	培养瓶查询、母瓶更正、株数更正、可用母瓶与子瓶筛选	/api/bottle/changeParentBottle
TissueTaskAction	16	任务创建、修改、打码、开始、完成、撤销、删除与查询	/api/tissueTask/createTask
StatAction	11	库存简报、库存统计、库存流水、预警、绩效统计与流水	/api/stat/inventorySummary
BottleCodeComplementTaskAction	8	补码任务的建单、打码、核对与办结	/api/bottleCodeComplementTask/createTask
InventoryAddBottleAction	8	增瓶任务的建单、打码、核对与办结	/api/inventoryAddBottle/createTask
UserAction	6	员工维护、口令修改、登录与当前用户信息	/api/user/updatePassword
InventoryAdjustmentAction	5	调差单建单、明细录入、办结与查询	/api/inventoryAdjustment/done
VarietyAction / VarietyCodeAction	4 + 4	品种与品种编号的维护、启停与查询	/api/variety/queryAllEnabled
CodePrintAction	3	打印记录落库、补打与打印明细查询	/api/codePrint/save
BottleCodeSequenceAction	2	瓶码前缀申请与序列查看	/api/bottleCodeSequence/next
StorageInventoryBaselineAction / WorkloadBaselineAction	2 + 2	库存基线与绩效基线的维护与查询	/api/storageInventoryBaseline/save
PassportAction	1	登录换取令牌	/api/passport/login

3.3 业务层设计

业务层是设计的重心。每个 Manager 对应一个业务域，方法名即业务动作名，方法体内按“取当前用户—校验单据状态—校验前置条件—写库—记日志”的顺序编排。跨域协作只在 Manager 之间发生，不出现 Action 直接调用另一个 Action 的情况。

表 7 主要业务类与职责

类	核心方法	职责要点
TissueTaskManager	createTask / modifyTask / codeTask / continueCodeTask / startTask / finishTask / cancelFinish / deleteTask	任务全生命周期；创建时按工序校验计量字段，非诱导任务自动生成出库申请单；完成后校验作业组与附件
TissueTaskManager	acceptStorageOut / rejectBizForIn / inDoneForTask	任务侧的出库接收、入库驳回与入库办结；接收集合必须与明细数量一致
StorageInOutManager	outProcessForTask / outDoneForTask / rejStoreForOut / rejBizForIn / inspectionOut / acclimatizationOut	库房侧的备出、退回与质检炼苗出入库；逐瓶校验可用性与数量一致性
BottleManager	changeParentBottle / changePlantCount / queryParents / queryChildren	母瓶更正与株数更正，代际关系重建，上下追溯
BottleCodeSequenceManager	next / queryForPeek	前缀发号；next 使用悲观锁，超限抛溢出异常
InventoryAddBottleManager	createTask / codeTask / checkBottle / done	账外实物入账：建单、打码、核对、办结生成正式台账
BottleCodeComplementTaskManager	createTask / codeTask / checkBottle / done	缺码瓶补码；空母瓶为合法起点
InventoryAdjustmentManager	create / saveDetail / done	盘点调差；明细瓶数取 −1、0、1，株数有符号
PassportManager	login / getUserByToken / logout	令牌签发与校验，登录态驻留内存
StatManager	库存简报、统计、流水、预警、绩效	只读聚合，按品种、代数、时间维度统计

3.4 持久层设计

持久层统一使用 EntityManager 拼装 CriteriaQuery，按条件对象动态追加谓词，不依赖方法名派生查询的长命名。查询分两类：单表条件查询与投影聚合。前者返回实体列表并配合分页对象，后者返回统计结果，避免把大表实体全部加载进内存。

分页模型固定为两个对象：请求侧 QueryCondition 携带 condition 与 page，响应侧 QueryResultPaged 携带 list 与 page，页码信息取自公共库 com.chenhuiran.commons.lang.models.Page。所有列表接口共用该模型，前端因此只需一套分页控件。

批量写入设有统一约定：待写实体用 persist 挂入上下文，累计到 500 条即 flush 并 clear，再继续下一批。该约定用于任务打码、逐瓶入库、补码核对等一次性写入数百至数千条的场景，目的不是提速，而是把一级缓存规模压在可控范围内。

3.5 缓存与序列

缓存只承担两类数据：变更频率低且读多写少的字典项，以及在线令牌。缓存为进程内结构，无过期同步机制，重启即清空；因此任何依赖缓存的一致性要求都必须在业务层兜底，例如字典项在写入后立即失效对应键。

瓶码前缀序列是全局唯一性的来源。序列表保存当前计数值，next 方法先以悲观锁读取当前值，再自增并回写，锁的范围仅覆盖单行；超过 17^6−1（24 137 568）时抛出溢出异常并提示序列耗尽。查询场景走无锁只读方法，避免看板刷新阻塞发号。

3.6 统一异常与响应

异常处理把“可预期的业务失败”和“不可预期的运行故障”分开：前者带中文原文返回 400，直接呈现给操作员；后者返回 500 并附带错误编号，详细堆栈只进日志，避免把内部结构暴露到界面。

表 8 异常映射规则

异常类型	HTTP 状态	响应消息	使用场景
ErrorMessageException	400	异常自带的中文消息	业务校验不通过，例如“尚有未完成的作业组”
IllegalArgumentException / IllegalStateException	400	异常消息原文	参数非法、状态不允许
HttpException 401 / 403 / 404	401 / 403 / 404	对应语义的提示	未登录、无权限、资源不存在
其他运行时异常	500	系统业务异常、系统运行异常或系统未知异常，附错误编号	空指针、数据库错误、外部依赖失败

前端对 400 与 500 的处置不同：400 直接弹出消息并把焦点定位到出错字段；500 弹出统一提示并提示记录错误编号，避免操作员反复重试造成重复提交。

3.7 事务与并发

事务边界设在 Manager 的写方法上，一次业务动作对应一个事务。典型的多表写入包括：打码（任务状态 + 瓶记录 + 打印明细）、备出（单据状态 + 明细实际数 + 瓶仓储状态 + 日志）、入库办结（单据状态 + 瓶台账 + 库存流水 + 绩效流水 + 日志）。

这些写入要么全部成功，要么全部回滚，不允许出现“单据已办结但流水未生成”的中间态。

并发控制采取两条措施。其一是序列发号的悲观锁，保证前缀不重号；其二是状态迁移前的条件校验，即更新时以“当前状态”作为条件之一，使并发下的第二次提交因状态不匹配而失败，而非覆盖前一次结果。系统按单实例部署，未引入分布式锁。

3.8 状态机实现

任务与出入库各有一套状态机，实现方式一致：状态以字符串常量集中定义在 Const 中，迁移入口是一个独立方法，方法首行校验当前状态，随后校验业务前置条件，最后一次性写入新状态与关联数据。

表 9 任务状态迁移与校验点

迁移	方法	校验点	副作用
— → created	createTask	批次为 0 或 YYYYMMDD；诱导必填预计母株数；其余工序必填预计子瓶数与子株数	非诱导任务生成出库申请单与申请日志
created → coded	codeTask	瓶数 1～9999；前缀已分配且未被占用；序号从 1 连续编	写瓶记录（子瓶、已打码）、写打印明细、任务置已打码
coded → coded（续打）	continueCodeTask	剩余容量 ≥ 本次瓶数，否则提示最多还能打印 N 个	追加瓶记录与打印明细，序号顺延
created/coded → created/coded	modifyTask	仅未开始或已打码可改；执行中需管理员强制编辑	变更预计产量与作业组配置
coded → processing	startTask	作业组已分配母瓶；记录株数合法	任务置执行中，允许录入生产记录
processing → finished	finishTask	无未完成作业组；诱导任务须上传执行任务附件	任务置完成，可发起入库
finished → processing	cancelFinish	该任务下无任何入库（含已办结）	回退状态，允许补录
created/coded → 删除	deleteTask	无已开始的出库或入库业务	记录删除，瓶码前缀不再复用

4 数据设计

4.1 数据模型总览

数据模型分为五组：基础参数、组培任务、出入库业务、库存与绩效、追溯与附件。分组不是物理分区，而是读写特征的归纳——基础参数读多写少，任务与出入库是事务主体，库存与绩效是只增不改的流水，追溯与附件按需写入。



图 2 数据表分组与主数据流

表间关系以三条主线贯通：任务派生作业组、作业组绑定瓶；出入库单派生明细、明细指向瓶；瓶的每一次状态变化写入瓶日志与库存流水。任何一条主线断开，追溯即失去依据，因此外键虽未在库层强制，业务层对引用的校验不可省略。

4.2 数据表清单

表 10 数据表分组

分组	表	说明
基础参数	variety、variety_code、user、dictionary、dictionary_item、storage_inventory_baseline、workload_baseline	品种与编号、员工与角色、字典、两类基线
组培任务	tissue_task、tissue_task_opp_group、tissue_task_bottle、archive	任务头、作业组、任务内每瓶的角色与株数、任务附件
出入库业务	storage_in_out、storage_in_out_bottle、storage_in_out_log	单据头、逐瓶明细、状态变更日志
库存与绩效	bottle、inventory_transaction、workload_transaction、inventory_adjustment、inventory_adjustment_detail	瓶台账、库存流水、绩效流水、调差单与明细
追溯与打印	bottle_relation、bottle_log、code_print、code_print_detail、bottle_code_sequence	代际闭包、瓶日志、打印记录与明细、前缀序列
辅助与临时	code_imp、inv_temp、inv_temp_1	导入与中间计算用，不参与正式口径

完整字段清单见附录 A。三张辅助表仅用于导入与中间核对，正式统计不得读取，避免出现“同一指标两个来源”的口径分裂。

4.3 核心表结构

核心表的字段设计有两处值得说明：瓶表同时保存“当前快照”与“末次任务快照”，使常用查询不必回连任务表；出入库明细保留“申请数”与“实际数”两组字段，使备出过程中的差异留痕，而不必事后靠日志还原。

表 11 核心表关键字段

表	关键字段	设计说明
tissue_task	id、task_type、id_variety、id_variety_code、batch、estimated_parent_plants、estimated_children_bottles、estimated_children_plants、code_prefix、status、force_edit、create_time、start_time、finish_time	任务头；code_prefix 为全局瓶码前缀，status 承载状态机
tissue_task_opp_group	id、id_task、parent_code、parent_plants、keep_flag、keep_plants、id_operator、id_checker、plan_plants、actual_plants、assign_time、finish_time	作业组；一个作业组绑定一个母瓶与一名操作员、一名质检员
tissue_task_bottle	id_task、id_opp_group、id_bottle（联合主键）、role_in_task、plant_count_plan、plant_count_actual、operate_status、child_code_status、parent_source_type	任务内每瓶的角色与株数；联合主键保证一瓶在一组内唯一
bottle	id、code、id_variety、id_variety_code、batch、generation、plant_count、source_type、parent_code、storage_status、operate_status、check_status、discard_flag、id_task_current、id_task_last	瓶台账；快照字段用于列表与追溯，避免连表
bottle_relation	id、child_id、parent_id、depth	代际闭包；depth 为 0 表示自身，1 为父瓶，支持任意层追溯
storage_in_out	id、in_out、bottles_applied、bottles_actual、plants_applied、plants_actual、keep_bottles、keep_plants、discard_plants、reason、status、id_task、id_check_record、id_applicant、id_operator	出入库单；申请数与实际数分列，out 记负、in 记正
storage_in_out_bottle	id、id_in_out、id_bottle、bottle_type、plant_count_applied、plant_count_actual、status、discard_reason、id_checker	明细；bottle_type 决定该瓶在本次业务中的角色
inventory_transaction	id、biz_type、id_source、id_variety、id_variety_code、generation、bottles、plants、id_operator、create_time	库存流水；有符号数量，所有报表由流水求和
workload_transaction	id、id_user、role_type、biz_type、id_source、id_variety、generation、bottles、plants、create_time	绩效流水；按操作员与质检员分别记账

4.4 编码与枚举

系统不使用数据库枚举类型，所有枚举以字符串代码存放，含义由字典表与常量类共同维护。该方式下库层不校验取值，新增取值不需改表结构，日志可直接读出业务含义。

表 12 主要编码取值

字段	取值	说明
task_type / storage_reason	induction、differentiation、multiplication、rooting、acclimatization	五类工序，兼作出入库原因
任务状态 status	created、coded、processing、finished、canceled	canceled 在流程中无迁移入口
出入库状态 status	apply、started、done、finished、rej_biz、rej_store、canceled、checked	checked 无迁移入口
storage_in_out_bottle.bottle_type	parent_out、child_in、parent_in、parent_keep_in、intercept_out、inventory_add、inventory_sub	单据明细的业务类别，决定流水方向
bottle.operate_status	none、coded、assigned、complement、finished、discard、inventory_add、inventory_sub、voided	瓶的操作状态，与仓储状态正交
bottle.storage_status	in、out	仓储状态，仅表示是否在库
bottle.role_in_task	none、parent、child	瓶在某任务中的角色
user.role	admin、task、inventory、task_and_inventory、query、none	六类角色，菜单与接口双重控制
in_out	in、out	出入库方向，配合有符号数量使用

4.5 键、约束与索引

主键统一使用 32 位字符串，由服务端生成，避免自增主键在数据迁移与合并时的冲突。业务唯一性由联合唯一索引承担：瓶码全局唯一，任务瓶以任务、作业组、瓶三列联合唯一，单据明细以单据与瓶联合唯一。

表 13 主要索引设计

表	索引	用途
bottle	uk_code（code）、idx_variety_gen（id_variety, generation）、idx_storage（storage_status）	扫码定位、按品种代数筛选、在库瓶查询
tissue_task	idx_status（status）、idx_create（create_time）、uk_prefix（code_prefix）	任务看板、按时间统计、前缀唯一
tissue_task_bottle	pk（id_task, id_opp_group, id_bottle）、idx_bottle（id_bottle）	组内逐瓶操作、按瓶反查任务
storage_in_out	idx_status（status）、idx_task（id_task）、idx_create（create_time）	待办单据、任务关联、时段统计
storage_in_out_bottle	uk（id_in_out, id_bottle）、idx_bottle（id_bottle）	明细唯一、按瓶查史
inventory_transaction	idx_biz_time（biz_type, create_time）、idx_variety（id_variety, generation）	库存统计与流水查询的主要入口
workload_transaction	idx_user_time（id_user, create_time）、idx_biz（biz_type）	绩效统计按员工与时段聚合
bottle_relation	uk（child_id, parent_id）、idx_parent（parent_id, depth)	向上向下追溯
archive	idx_target（table_name, record_id）	按业务记录取附件

4.6 冗余与水化

系统有三处冗余，各自在写入时由业务层同步维护。

- 瓶表快照：bottle 保存当前任务与末次任务的编号、状态，任务变更时由业务层同步刷新。瓶列表与追溯页因此不回连任务表。
- 闭包表：bottle_relation 为每对祖先—后代写一行，每对祖先—后代写一行，任意层级追溯为一次索引扫描，无需递归查询。
- 基线表：库存基线与绩效基线把期初数字固化，使“期初 + 入 − 出 + 调差”的等式成立。报表由期初、入、出、调差四项复算；基线变更时重算期间数据。

三处冗余在任务删除、瓶码作废、母瓶更正时由业务层成对更新：快照随任务变更刷新，闭包随母瓶更正重建，基线变动时重算期间数据。

4.7 容量与大表治理

按单任务 100～2000 瓶、年任务 2000～3000 个估算，瓶表与流水表是增长最快的三张表，三年内达到百万至千万行量级。系统未引入分库分表与分区表，增长由索引与归档控制。

表 14 增长预估与治理策略

表	年增量	现有处理方式
bottle	20 万～60 万	按批次与代数建复合索引；出库瓶不删除，仅改状态；历史批次按年导出归档
bottle_relation	瓶数的 3～6 倍	闭包按深度展开写入；母瓶更正时重建受影响子树
storage_in_out / _bottle	30 万～90 万	单据与明细按时间归档；列表查询带时段条件
inventory_transaction / workload_transaction	30 万～90 万	只增不改；报表按区间聚合，明细供追溯
tissue_task_bottle	20 万～60 万	随任务生命周期增长，随任务归档

5 接口设计

5.1 协议与地址规范

所有业务接口以 HTTP POST 暴露，请求与响应均为 JSON，字符集 UTF-8；附件上传使用表单提交，下载与导出返回二进制流。地址统一为“/api/模块/动作”，模块名与控制器同名的小驼峰形式，动作名为动词性短语，例如 /api/tissueTask/codeTask。

该规范带来的确定性有三方面：参数不受 URL 长度限制；中文与特殊字符无需二次编码；拦截器与日志可对全部接口一视同仁。幂等性由业务层保证，接口层不做重复提交拦截。

5.2 请求与响应模型

表 15 通用模型

模型	字段	说明
QueryCondition<C>	condition、page	列表查询统一入参；condition 为各业务自定义筛选对象
QueryResultPaged<T>	list、page	列表查询统一出参；page 含页码、页大小与总数
Page	pageNumber、pageSize、total	公共库分页对象，前后端共用
业务响应	业务对象或布尔值	成功时直接返回结果体，不额外包一层 code/message
错误响应	message	失败时由异常处理器输出中文消息，配合 HTTP 状态码

响应不套统一外壳是有意为之：成功即数据、失败即状态码，前端拦截器据状态码分流，避免“HTTP 200 但业务失败”的隐性错误。前端在每个调用点处理非 2xx 分支，该分支集中在数据层统一实现。

5.3 鉴权与权限

登录成功后服务端返回令牌，前端保存在本地存储并由拦截器附加到每个请求的 x-ptcmis-token 头。AuthInterceptor 依此取出用户；标注 @Permission(value = Const.USER_ROLE_NONE) 的方法跳过令牌校验，仅限登录与少量公共查询。

角色校验采用注解加常量的方式：方法上声明允许的角色集合，拦截器或切面在进入方法前比对当前用户角色，不匹配即返回 403。角色以常量硬编码在代码中，新增角色需改动代码并重新发布。

5.4 错误码与提示

系统未定义数字错误码表，错误以“HTTP 状态 + 中文消息”表达。业务校验消息要求写明对象与原因，例如“瓶码 ABCD-0001 不在库，无法出库”，不允许出现“操作失败”这类无信息量提示。运行异常另附错误编号，便于按日志定位。

5.5 主要接口

下表列出承载关键业务的接口，完整 116 个端点见附录 B。接口清单按“动作—入参要点—出参要点—约束”组织，便于前后端对齐。

表 16 主要接口清单

接口	入参要点	出参	约束
/api/passport/login	账号、口令	令牌、用户信息与角色	口令明文比对，见第 8 章
/api/tissueTask/createTask	工序、品种、编号、批次、预计产量	任务对象	按工序校验计量字段；非诱导自动建出库申请
/api/tissueTask/codeTask	任务、瓶数	打印明细与瓶记录	瓶数 1～9999；前缀唯一
/api/tissueTask/startTask	任务、作业组与母瓶分配	任务对象	作业组必须绑定在库母瓶
/api/tissueTask/finishTask	任务、作业组完成情况	任务对象	无未完成作业组；诱导任务需附件
/api/storageInOut/outProcessForTask	单据、逐瓶实际株数	单据与明细	逐瓶校验在库；集合必须与明细一致
/api/tissueTask/acceptStorageOut	单据、接收集合	单据状态	接收数量必须等于明细数
/api/storageInOut/inDoneForTask	单据、逐瓶入库信息	单据与流水	任务须完成；四类入库分流
/api/bottle/queryParents、queryChildren	瓶码、深度	瓶列表	闭包表查询，深度受限
/api/stat/inventorySummary	品种、时段	汇总指标	只读，走流水聚合
/api/codePrint/save	任务、明细、打印结果	打印记录	失败也落库，便于补打

5.6 打印通道

标签打印有两条通道：浏览器插件通道经 LODOP 直接驱动打印机，适合已部署控件的工位；本地进程通道经 HTTP 或标准输入输出与 ptcmis-printer 通信，适合需要统一分发、免装插件的场景。两条通道共用同一份打印明细数据，切换不改变业务记录。

无论走哪条通道，打印结果都回写 code_print 与 code_print_detail，记录打印机、模板、成功与失败原因。回写是补打与对账的依据，因此前端在打印完成后必须调用回写接口，不能仅以“已发送”视为成功。

5.7 扫码、附件与导出

扫码以键盘模拟方式接入：前端监听全局按键，以 80 毫秒静默间隔判定一次扫码结束，把累计字符注入当前焦点输入框并触发查询。该方式无需驱动与中间件，但要求扫码枪配置为回车结尾，且不与中文输入法冲突。

附件上传按业务表名与记录主键建立目录，文件名与原始名分列保存，下载时以原始名回写响应头。导出走预置模板填充，返回二进制流，文件名含报表名与导出时间。

6 前端设计

6.1 技术形态

前端以 Angular 22 的独立组件（standalone）组织，采用无 zone.js 的变更检测（zoneless）与信号量（signal）状态，配合 Angular Material 提供表格、对话框与表单控件，图表使用 ECharts 6。构建产物为静态资源，由 Nginx 托管并对 SPA 路由做回退。

组件刷新由显式写入信号量触发：异步回调结束后须主动更新状态，系统不依赖隐式脏检查。

6.2 目录结构

表 17 前端目录与职责

目录	内容	职责
src/screens	35 个界面目录（00100-welcome 至 04300-query-bottle）	按业务编号排序的页面，目录名即界面标识
src/services/datas	各业务数据服务与 api 拦截器	封装接口调用、参数整形、错误分流与令牌注入
src/services/actions	动作层服务	把多步数据调用编排为一个业务动作
src/entities	前端实体与模型	与后端模型对应的类型定义
src/shared	公共组件、管道、校验器	表格、扫码输入、确认对话框等复用件
src/app	根组件、路由与主题	路由表、菜单装配与全局样式

目录编号而非中文名，是为了让菜单顺序、权限配置与代码位置保持一一对应：00100 为登录与欢迎，04100 起为统计，04300 为培养瓶查询。新增界面按编号区间插入，不打乱既有顺序。

6.3 数据层设计

数据层分两级：datas 目录下的服务负责单接口调用，actions 目录下的服务负责多接口编排。前者只关心入参与出参，后者关心业务顺序与失败回滚，例如“创建任务后立即打码”会落在 actions 中，而不是页面组件里。

api 拦截器承担三件事：附加 x-ptcmis-token；统一处理 401（清空登录态并跳转登录）；把 4xx 的中文消息转为可展示的错误对象。拦截器不重试、不缓存，失败即抛给调用方，由调用方决定提示或回退。

6.4 界面清单

35 个界面归入六个模块，模块划分与后端控制器并不严格一一对应：任务与库存共用作业台，统计与参数各自成组。下表给出模块、界面数与代表界面。

表 18 界面模块与代表界面

模块	界面数	代表界面	说明
公共	4	登录、首页、我的账号、打印设置	登录态、菜单与本地打印配置
组培任务	4	诱导、分化、生根、炼苗	四类工序共用任务工作台，按 task_type 参数化
库存管理	11	瓶体补码、任务出库、任务入库、质检出库、炼苗出入库、盘点调差	按单据与逐瓶操作组织，扫码为主要输入方式
标签打印	2	打印记录、补打瓶码	打印结果查询与失败重打
查询统计	7	库存简报、库存统计、库存流水、库存预警、绩效统计、绩效流水、培养瓶查询	只读，支持 Excel 导出
系统参数	5	品种信息、品种编号、员工信息、库存基线、绩效基线	主数据维护，仅管理员可见

6.5 交互与校验约定

界面遵循三条交互约定：其一，所有列表页支持按品种、编号、批次、代数与时间组合筛选，筛选条件参与分页；其二，危险动作（删除任务、撤销完成、驳回、更正母瓶）必须二次确认并展示影响范围，例如“将影响 12 个作业组”；其三，表单校验在失焦时即时提示，提交时整体校验，错误定位到字段。

扫码输入的焦点策略在 6.5 中单独约定：每次扫码成功后自动聚焦下一输入项，连续扫码场景（备出、核对、补码）不允许弹出模态框阻断流程；若扫码结果不匹配，则以红底提示并保持焦点原地，便于重扫。

6.6 打印集成

打印设置页保存打印机名称，写入本地存储；打印时按设置组装指令。选择 LODOP 通道时由控件负责渲染与发送，选择本地进程通道时由前端调用 ptcmis-printer 并将返回的 JSON 结果解析为提示。两条通道的差异被封装在打印服务内，页面不感知。

打印失败的处置统一为：保留明细、记录原因、允许补打。补打走同一接口但携带原打印记录标识，避免重复生成瓶码。

6.7 构建与性能

构建使用 ng build 输出静态资源，Nginx 启用 gzip 与 7 天静态缓存，index.html 不缓存以保证版本更新即时生效。界面按路由懒加载，统计页的 ECharts 按需引入，避免首屏加载完整图表库。

列表与扫码的性能依靠分页与条件收窄实现，未引入前端虚拟滚动；列表首屏目标 2 秒内、扫码到结果 1 秒内。

7 打印子系统设计

7.1 进程与通信模型

ptcmis-printer 是一个常驻的本地进程，由操作系统启动后监听本地通道，按行读取 JSON 请求、按行写回 JSON 响应。进程无状态，不缓存任务与瓶数据，每次请求自带全部打印内容，因此重启不影响业务，也不会因为缓存过期打错标签。

通信采用标准输入输出上的 JSON 行协议：一行一个请求，一行一个响应，请求与响应通过序号对应。选择行协议而非 HTTP 服务，是为了让宿主程序（浏览器插件、桌面壳或命令行）以最简单的方式拉起与管控进程，同时避免占用端口带来的防火墙问题。

7.2 报文约定

表 19 打印进程报文

字段	类型	说明
action	字符串	动作名，取值 get_printers、get_printer_status、add_job
params	对象	动作参数；add_job 时含打印机、模板与内容
content	对象数组	标签内容：文本行、二维码、条码与坐标
success	布尔	处理结果
data	对象	成功时的返回数据，例如打印机列表或作业标识
error	对象	失败时的错误码与中文描述

报文设计遵循“一次请求完成一件事”：查询打印机、查询状态、提交作业各自独立，失败可单独重试。不在一次请求中混合查询与打印，避免部分成功难以判定。

7.3 指令装配

标签内容最终转为 TSPL 文本指令，按标签尺寸、间隙、方向依次下发，再逐条下发文本与二维码。装配职责在打印进程内，业务系统只提供语义化的内容（例如“品种：XX”“瓶码：ABCD-0001”），不拼指令。标签版式调整只需改动打印进程。

表 20 标签指令要素

要素	指令	说明
标签尺寸	SIZE	宽高以毫米为单位，40×20 为常用规格
间隙与黑标	GAP / BLINE	按纸型选择，连续纸与间隙纸不可混用
方向	DIRECTION	0 或 180，取决于走纸方向与操作视角
浓度与速度	DENSITY / SPEED	按打印机与碳带调校，作为本地配置而非业务参数
文本	TEXT	按字体、坐标、内容逐行下发，中文需声明编码
二维码	QRCODE	含容错等级与模块尺寸，瓶码采用较高容错以适配磨损
打印	PRINT 1,1	一次性提交，份数固定为 1，重打由业务层控制

7.4 平台适配

表 21 平台适配要点

平台	接口	要点
Windows	WinSpool	按打印机名称取设备上下文，以 RAW 方式提交指令流；需处理驱动缺纸与离线状态
Linux	CUPS	通过 CUPS 队列提交，打印机名即队列名；需注意编码与权限
公共	配置读取	打印机名、纸型、浓度来自本地配置，不由服务端下发

7.5 错误处理

错误分为可重试与不可重试两类：打印机离线、缺纸、超时属可重试，返回明确原因供操作员处理后重打；指令非法、模板缺字段属不可重试，需修正数据。两类错误都以中文描述返回，业务端不解析错误码做分支，只把描述呈现给操作员。

表 22 典型错误

场景	表现	处置
打印机不存在	按名称取不到设备	提示核对打印机名与驱动，进入打印设置
打印机离线	状态查询返回离线	提示检查电源与连接，恢复后重打
缺纸或卡纸	作业提交后状态异常	提示处理纸路，保留明细待补打
指令超时	提交后长时间无完成回执	判定失败并记录，避免重复提交
模板缺字段	内容装配阶段校验失败	提示缺失字段，修正后重试

7.6 构建与分发

工程采用 CMake 3.18 以上与 C++17 标准，Windows 与 Linux 分别编译，产物为单文件可执行程序，不依赖运行时库以外的第三方组件。构建脚本不捆绑前端资源，打印进程与前端通过协议解耦，可独立升级。

分发时随附配置样例与启动说明；升级只替换可执行文件，配置文件保留。由于进程无状态，升级不需要停机窗口，重启后下一次打印即生效。

8 安全设计

8.1 认证与会话

认证采用“账号口令换取令牌”的方式：登录成功后服务端生成随机令牌并与用户绑定，保存在进程内存；后续请求凭 x-ptcmis-token 识别身份。令牌不落库、不落盘，服务重启即全部失效。

该方案有两处实现特征：令牌无过期时间，仅在退出登录或服务重启时失效；令牌驻留服务进程内存，不跨实例共享。

8.2 授权模型

授权分两层：菜单层由前端按角色装配可见项，接口层由权限注解校验。两层不是重复，而是分工——前端控制“看得见”，后端控制“做得到”。越权访问的兜底在后端：即使绕过界面直接调用接口，注解仍会拦截。

表 23 角色与授权范围

角色	可访问模块	典型限制
admin	全部	员工维护、品种与编号维护仅管理员可写
task	组培任务、标签打印、查询统计	不可操作出入库单据
inventory	库存管理、标签打印、查询统计	不可创建或修改任务
task_and_inventory	任务与库存	兼具两侧权限，用于小型工位
query	查询统计	只读，写接口一律 403
none	个人账号	仅登录与改密

8.3 口令与令牌的存储

员工口令以明文存放于 user 表的 password 字段（VARCHAR(16)），登录时直接字符串比对；令牌由服务端随机生成后仅存于进程内存，不落库、不落盘。

8.4 输入校验与防护

输入校验分三层：前端表单在失焦与提交时校验，接口层完成类型与范围绑定，业务层完成语义校验。

表 24 校验与异常返回的现有实现

场景	现有实现
查询构造	经 CriteriaQuery 与参数绑定组装，不拼接 SQL 字符串
权限校验	权限注解加令牌校验，写接口逐一声明角色
附件落盘	按业务主键建立目录，原始名与存储名分列保存
附件下载	按记录主键拼路径，不接受前端传入的相对路径
重复提交	以当前状态作为更新条件，状态不符即失败
异常返回	统一处理：400 回中文原文，500 只回错误编号，堆栈只进日志

8.5 审计与日志

审计依赖三类记录：bottle_log 记录瓶的每次状态变化，storage_in_out_log 记录单据状态变更与驳回原因，应用日志记录请求、耗时与错误编号。三者结合可以回答“谁在何时把哪瓶改成了什么”。

日志保留策略为业务日志不少于三年、运行日志不少于三十天并滚动清理。日志中不写口令与完整令牌，只写用户标识与令牌摘要。

9 部署与运维

9.1 部署拓扑

表 25 部署单元

单元	部署位置	端口	说明
ptcmis-web	应用服务器（Nginx）	9006	静态资源与 SPA 回退，gzip 与 7 天缓存
ptcmis-server	应用服务器（JVM）	9005	单实例部署，令牌驻留内存
MySQL	数据库服务器	9007	库 ptcmis，字符集 GB18030，连接池 10～20
ptcmis-printer	车间客户端	—	本地进程，随工位安装，无网络监听
浏览器	车间与办公终端	—	Chrome 或 Edge 近两年版本，分辨率不低于 1920x1080

9.2 配置项

后端配置集中在 application.yaml，包含端口、数据源、字符集与连接池；前端配置集中在 Nginx 的 nginx.conf，包含反向代理、gzip 与缓存策略；打印进程配置为本地文件，包含打印机名、纸型与浓度。三处配置均不入库，变更需重启对应进程。

表 26 关键配置

配置	位置	取值	说明
服务端口	application.yaml	9005	后端监听端口
数据源	application.yaml	jdbc:mysql://…/ptcmis	连接串含字符集参数
字符集	application.yaml / 建库脚本	GB18030	库、表、连接三处一致
批量写入	业务代码常量	500	每 500 条 flush 与 clear
前端端口	nginx.conf	9006	静态托管与代理
缓存时长	nginx.conf	7 天	静态资源缓存，index.html 除外
令牌头	前后端约定	x-ptcmis-token	改名需前后端同步修改
扫码间隔	前端常量	80 毫秒	判定一次扫码结束的静默时长

9.3 构建与发布

后端以 Maven 打出可执行 JAR，前端以 ng build 产出静态目录，打印进程以 CMake 产出可执行文件。发布顺序为：先发布数据库变更（如有），再发布后端，最后发布前端与打印进程。前端与后端解耦，可独立回滚；数据库变更需提供回滚脚本。

9.4 备份与恢复

数据库每日本地全量备份，保留 7 天。

恢复顺序为：先恢复库结构与基础参数，再恢复业务数据。

9.5 监控与排障

监控以可用性与错误率为核心：服务进程存活、接口 5xx 比例、数据库连接数与慢查询、打印失败率。排障依赖错误编号：前端提示编号，运维按编号在日志中检索完整堆栈，避免在界面上暴露内部结构。

表 27 常见故障对照

现象	可能原因
页面可打开但接口 401	令牌失效或服务重启
列表查询超时	缺少时段条件或索引失效
打印无输出	打印机离线或名称不符
库存与报表不符	流水缺失或符号错误
中文乱码	连接字符集非 GB18030

10 设计约束

10.1 设计约束

以下约束由现有实现决定，突破时会改变既有数据结构、接口约定或部署方式。

- 单实例：令牌与缓存均在进程内，部署为单个 JVM 进程。
- 单库：不使用分库分表，增长由索引与归档控制。
- 令牌头固定：x-ptcmis-token 同时写在前端拦截器与后端拦截器，改名需同时发布。
- 接口全 POST：全部端点以 POST 暴露。

- 瓶码不可复用：前缀与瓶码一经分配即永久占用，作废只改状态不回收编号。
- 打印在客户端：服务端不承担打印渲染，版式由打印进程与前端设置决定。

附录 A 数据表清单

下表按 PO 注释解析所得，列出 32 张表的表名、对应实体类与字段数。字段级明细含列名、DDL 类型与默认值，见源码 stubs/entities 下的注释与 baseline.sql。

表 28 数据表清单

序号	表名	实体类	字段数
1	archive	Archive	11
2	bottle_code_complement_task_detail	BottleCodeComplementTaskDetail	10
3	bottle_code_complement_task	BottleCodeComplementTask	18
4	bottle_code_sequence	BottleCodeSequence	2
5	bottle_log	BottleLog	16
6	bottle	Bottle	48
7	bottle_relation	BottleRelation	4
8	code_imp	CodeImp	1
9	code_print_detail	CodePrintDetail	7
10	code_print	CodePrint	7
11	dictionary_item	DictionaryItem	3
12	dictionary	Dictionary	2
13	inv_temp_1	InvTemp1	1
14	inv_temp	InvTemp	1
15	inventory_add_bottle_detail	InventoryAddBottleDetail	10
16	inventory_add_bottle	InventoryAddBottle	18
17	inventory_adjustment_detail	InventoryAdjustmentDetail	5
18	inventory_adjustment_log	InventoryAdjustmentLog	6
19	inventory_adjustment	InventoryAdjustment	11
20	inventory_transaction	InventoryTransaction	13
21	storage_in_out_bottle	StorageInOutBottle	13
22	storage_in_out_log	StorageInOutLog	7
23	storage_in_out	StorageInOut	26
24	storage_inventory_baseline	StorageInventoryBaseline	5
25	tissue_task_bottle	TissueTaskBottle	22
26	tissue_task_opp_group	TissueTaskOppGroup	19
27	tissue_task	TissueTask	20
28	tissue_task_user	TissueTaskUser	3
29	user	User	7
30	variety_code	VarietyCode	4
31	variety	Variety	2
32	workload_transaction	WorkloadTransaction	16

附录 B 接口清单

下表列出全部 116 个端点，按控制器分组。地址为“类级前缀 + 方法级路径”的拼接结果，均为 POST。

表 29 接口清单

序号	控制器	接口路径
1	Bottle	/api/bottle/queryByCondition
2	Bottle	/api/bottle/queryAllByCondition
3	Bottle	/api/bottle/changeParentBottle
4	Bottle	/api/bottle/changePlantCount
5	Bottle	/api/bottle/queryAllEnabledIdsForTaskParents
6	Bottle	/api/bottle/queryAllEnabledIdsForCodeComplementParents
7	Bottle	/api/bottle/queryAllEnabledIdsForInventoryAddBottle
8	Bottle	/api/bottle/queryAllEnabledParentIdsForChangeParent
9	Bottle	/api/bottle/queryAllStorageIn
10	Bottle	/api/bottle/queryBottlesForInventorySubBottle
11	Bottle	/api/bottle/queryAllForAcclimatizationIn
12	Bottle	/api/bottle/queryById
13	Bottle	/api/bottle/queryAllIdStorageIn
14	Bottle	/api/bottle/queryParents
15	Bottle	/api/bottle/queryChildren
16	Bottle	/api/bottle/queryLogs
17	Bottle	/api/bottle/queryArchives
18	Bottle	/api/bottleexcelBottle
19	BottleCodeComplementTask	/api/bottle-code-complement-task/queryByCondition
20	BottleCodeComplementTask	/api/bottle-code-complement-taskqueryOrCreateCodePrefix
21	BottleCodeComplementTask	/api/bottle-code-complement-task/queryAllCodesInTask
22	BottleCodeComplementTask	/api/bottle-code-complement-task/save
23	BottleCodeComplementTask	/api/bottle-code-complement-task/delete
24	BottleCodeComplementTask	/api/bottle-code-complement-task/printCode
25	BottleCodeComplementTask	/api/bottle-code-complement-task/process
26	BottleCodeComplementTask	/api/bottle-code-complement-task/done
27	BottleCodeSequence	/api/bottom-code-sequence/next
28	BottleCodeSequence	/api/bottom-code-sequence/peek
29	CodePrint	/api/code-print/queryByCondition
30	CodePrint	/api/code-print/queryAllDetails
31	CodePrint	/api/code-print/save
32	InventoryAddBottle	/api/inventory-add-bottle/queryByCondition
33	InventoryAddBottle	/api/inventory-add-bottlequeryOrCreateCodePrefix
34	InventoryAddBottle	/api/inventory-add-bottle/queryAllCodesInTask
35	InventoryAddBottle	/api/inventory-add-bottle/save
36	InventoryAddBottle	/api/inventory-add-bottle/delete
37	InventoryAddBottle	/api/inventory-add-bottle/printCode

序号	控制器	接口路径
38	InventoryAddBottle	/api/inventory-add-bottle/process
39	InventoryAddBottle	/api/inventory-add-bottle/done
40	InventoryAdjustment	/api/inventory-adjustment/queryByCondition
41	InventoryAdjustment	/api/inventory-adjustment/queryAllLogs
42	InventoryAdjustment	/api/inventory-adjustment/save
43	InventoryAdjustment	/api/inventory-adjustment/delete
44	InventoryAdjustment	/api/inventory-adjustment/done
45	Passport	/api/passportlogin
46	Stat	/api/statstatInventorySimple
47	Stat	/api/statstatInventory
48	Stat	/api/statstatInventoryTransaction
49	Stat	/api/statstatWorkload
50	Stat	/api/statstatWorkloadTransaction
51	Stat	/api/statexcelStatInventorySimple
52	Stat	/api/statexcelStatInventory
53	Stat	/api/statexcelStatInventoryAlert
54	Stat	/api/statexcelStatInventoryTransaction
55	Stat	/api/statexcelStatWorkload
56	Stat	/api/statexcelStatWorkloadTransaction
57	StorageInOut	/api/storage-in-out/queryAllLogs
58	StorageInOut	/api/storage-in-out/queryByCondition
59	StorageInOut	/api/storage-in-out/queryBottles
60	StorageInOut	/api/storage-in-out/rejBiz
61	StorageInOut	/api/storage-in-out/outProcessForTask
62	StorageInOut	/api/storage-in-out/outDoneForTask
63	StorageInOut	/api/storage-in-out/rejStore
64	StorageInOut	/api/storage-in-out/rejBizForIn
65	StorageInOut	/api/storage-in-out/inDoneForTask
66	StorageInOut	/api/storage-in-out/queryInspectOutByCondition
67	StorageInOut	/api/storage-in-out/createInspectOut
68	StorageInOut	/api/storage-in-out/outProcessForInspect
69	StorageInOut	/api/storage-in-out/outFinishForInspect
70	StorageInOut	/api/storage-in-out/deleteForInspect
71	StorageInOut	/api/storage-in-out/createInventorySubBottle
72	StorageInOut	/api/storage-in-out/outProcessForInventorySubBottle
73	StorageInOut	/api/storage-in-out/outFinishForInventorySubBottle
74	StorageInOut	/api/storage-in-out/deleteForInventorySubBottle
75	StorageInOut	/api/storage-in-out/createAcclimatizationOut
76	StorageInOut	/api/storage-in-out/outProcessForAcclimatizationOut
77	StorageInOut	/api/storage-in-out/outFinishForAcclimatizationOut
78	StorageInOut	/api/storage-in-out/deleteForAcclimatizationOut
79	StorageInOut	/api/storage-in-out/createAcclimatizationIn
80	StorageInOut	/api/storage-in-out/inProcessForAcclimatizationIn
81	StorageInOut	/api/storage-in-out/inFinishForAcclimatizationIn
82	StorageInOut	/api/storage-in-out/deleteForAcclimatizationIn
83	StorageInventoryBaseline	/api/storage-inventory-baseline/queryByCondition
84	StorageInventoryBaseline	/api/storage-inventory-baseline/save
85	TissueTask	/api/tissue-taskqueryByTaskId
86	TissueTask	/api/tissue-taskqueryByCondition
87	TissueTask	/api/tissue-taskqueryOrCreateCodePrefix
88	TissueTask	/api/tissue-taskcreateTask
89	TissueTask	/api/tissue-taskdeleteTask

序号	控制器	接口路径
90	TissueTask	/api/tissue-taskupdateTask
91	TissueTask	/api/tissue-taskprintBottleCode
92	TissueTask	/api/tissue-taskprintBottleCodeForAddBottles
93	TissueTask	/api/tissue-tasksaveProcessSettings
94	TissueTask	/api/tissue-taskdeleteStorageOut
95	TissueTask	/api/tissue-taskacceptStorageOut
96	TissueTask	/api/tissue-tasksaveProcess
97	TissueTask	/api/tissue-taskupdateTaskStatusToFinish
98	TissueTask	/api/tissue-taskupdateTaskStatusFromFinishToProcess
99	TissueTask	/api/tissue-taskapplyStorageIn
100	TissueTask	/api/tissue-taskdeleteStorageIn
101	User	/api/user/queryByIdByAdmin
102	User	/api/user/queryByCondition
103	User	/api/user/queryAllEnabled
104	User	/api/user/saveByAdmin
105	User	/api/user/toggleEnabled
106	User	/api/user/updatePassword
107	Variety	/api/variety/queryByCondition
108	Variety	/api/variety/queryAllEnabled
109	Variety	/api/variety/save
110	Variety	/api/variety/toggleEnabled
111	VarietyCode	/api/varietyCode/queryByCondition
112	VarietyCode	/api/varietyCode/queryAllEnabled
113	VarietyCode	/api/varietyCode/save
114	VarietyCode	/api/varietyCode/toggleEnabled
115	WorkloadBaseline	/api/workload-baseline/queryByCondition
116	WorkloadBaseline	/api/workload-baseline/save